



Maharaja Institute of Technology Mysore

Department of Computer Science & Engineering



Subject: Artificial Intelligence

Staff Name: Honnaraju B

Subject Code: 15CS562

SCHEME AND SOLUTION

Q. No.	MODULE 1	Marks									
1.	A Discuss the tic-tac-toe problem in detail and explain how it can be solved using AI technique. Answer: Tic-Tac-Toe Tic-tac-toe (also known as noughts and crosses or Xs and Os) is a paper-and-pencil game for two players, X and O, who take turns marking the spaces in a 3×3 grid. The player who succeeds in placing three of their marks in a horizontal, vertical, or diagonal row wins the game. The following example game is won by the first player, X: Players soon discover that the best play from both parties leads to a draw. Hence, tic-tac-toe is most often played by young children. Because of the simplicity of tic-tac-toe, it is often used as a pedagogical tool for teaching the concepts of good sportsmanship and the branch of artificial intelligence that deals with the searching of game trees. It is straightforward to write a computer program to play tic-tac-toe perfectly or to enumerate the 765 essentially different positions (the state space complexity) or the 26,830 possible games up to rotations and reflections (the game tree complexity) on this space. The programs in this series increase in: • Their complexity • Their use of generalizations • The clarity of knowledge • The extensibility of their approach. Thus, they move toward being representations of what we call AI techniques. PROGRAM 1 Data Structure: Board: A nine-element vector representing the board, where the elements of the vector correspond to the board position as follows: <table border="1" style="margin-left: auto; margin-right: auto;"> <tr> <td>1</td><td>2</td><td>3</td></tr> <tr> <td>4</td><td>5</td><td>6</td></tr> <tr> <td>7</td><td>8</td><td>9</td></tr> </table> An element contains the value 0 if the corresponding square is blank, 1 if it is filled with an X, or 2 if it is filled with an O. Movetable: A large vector of 19683 elements (3^9), each element of which is a nine-element vector. Algorithm: To make a move, do the following: 1. View the vector board as a ternary number (base three). Convert it to a decimal number. 2. Use the number computed in step 1 as index into Movetable and access the vector stored there.	1	2	3	4	5	6	7	8	9	10 2 2
1	2	3									
4	5	6									
7	8	9									

3. The vector selected in step 2 represents the way the board will look after the move that should be made. So set Board equal to that vector.

Advantages:

- This program is very efficient in terms of time,
- optimal game of tic-tac-toe in theory

Disadvantages:

- It takes a lot of space to store the table that specifies the correct move to make from each board position.
- Someone will have to do a lot of work specifying all the entries in the movetable.
- It is very unlikely that all the required Movetable entries can be determined and entered without any errors.
- If we want to extend the game, say to three dimensions - 3^{27} , no longer work at all.

PROGRAM 2

Data Structure:

Board: A nine-element vector representing the board, where the elements of the vector correspond to the board position as follows:

1	2	3
4	5	6
7	8	9

3

But instead of using the numbers, 0, 1, or 2 in each element, we store 2 (indicating blank), 3 (indicating X) or 5 (indicating O).

Turn: An integer indicating which move of the game is about to be played: 1 indicates the first move, 9 the last.

Algorithm: The main algorithm uses 3 *sub procedures*

Make2: Returns 5 if the center square of the Board [5] is blank, that is if Board [5] = 2. Otherwise return any blank non-corner square [2, 4, 6, or 8],

Posswin(p): Returns 0 if player p cannot win on his next move; otherwise it returns the number of the square that constitutes a winning move.

If product is 18 ($3*3*2$), then X can win.

If the product is 50 ($5*5*2$), then O can win.

- **Go (n):** Move to Board [n] to 3 if Turn is odd, or 5 if Turn is even. It also increments Turn by one.
- Turn=1 Go (1)
- Turn=2 Go (5) or Go (1)
- Turn=3 Go (9) or Go (3)
- Turn=4 Go(Posswin(X)) or Go(Make2)
- Turn=5 Go(Posswin(X)), or Go(Posswin(O)), or Go(7), or Go(3) [fork]
- ...

Comments

- Less efficient than Program 1 (time)
- More efficient (space)
- More clarity (strategy)
- Easier to change (strategy)
- Cannot extend to three dimension

PROGRAM 2'

This program is identical to program 2 except for one change in the representation of the board. The board positions to vector elements as follows:

8	3	4
---	---	---

1	5	9
6	7	2

Notice that this numbering of the board produces a magic square: all the rows, columns, and diagonals sum up to 15. This means that we can simplify the process of checking for a possible win. In addition to marking the board as moves are made, we keep a list, for each player, of the squares in which he or she has played.

S = sum of two paired owned by a player

D = 15 – S

if 0 < D < 10 and Board [D] is empty then the player can win

If this difference is not positive or if it is greater than 9, then the original two squares were not collinear and so can be ignored. Otherwise if the square representing the difference is blank, a move there will produce a win. Since no player can have more than four squares at a time, there will be many fewer squares examined using this scheme than there were using the more straightforward approach of program 2.

Comments:

This comparison raises an interesting question about the relationship between the way people solve problems and the way computer do. Why do people find the row-scan approach easier while the number-counting approach is more efficient for a computer? We do not know enough about how people work to answer that question completely.

PROGRAM 3

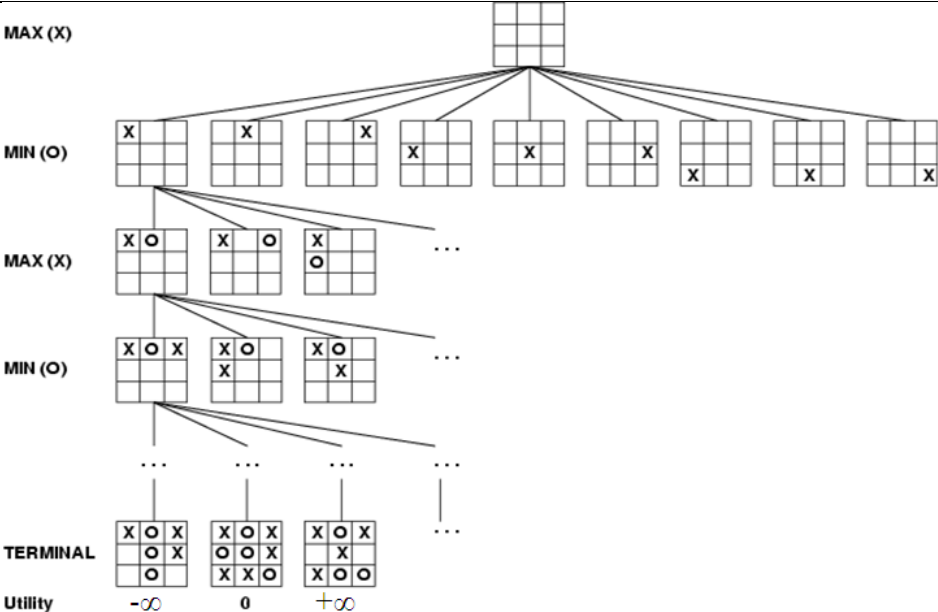
Data Structures

BoardPosition: A structure containing a nine-element vector representing the board, a list board positions that could result from the next move, and a number representing an estimate of how likely the board position is to lead to an ultimate win for the player to move.

The Algorithm: To decide on the next move, look ahead at the positions that result from each possible move. Decide which position is best, make the move that leads to that position, and assign the rating of that best move to the current position.

To decide which of a set of board positions is best, do the following for each of them:

1. See if it is a win. If so, call it the best by giving it the highest possible rating.
2. Otherwise, consider all the moves the opponent could make next. See which of them is worst for us. Assume the opponent will make that move. Whatever rating those move has, assign it to node we are considering.
3. The best node is then the one with the highest rating.

		 Comments - This program will require much more time than either of the others since it must search a tree representing all possible move sequences before making each move. - Extendable Solution: Define Tic-Tac Toe problem 2 Marks Explain program 1 2 Marks Explain program 2 2 Marks Explain program 3 4 Marks SCHEME [2 + 2 + 3 + 3 = 10]	
2.	A	B What do you mean by an AI Technique? List four things to build a system to solve a problem. Answer: Artificial Intelligence (AI) is the study of how to make computer do things which, at the moment, people do better. This definition is, of course somewhat ephemeral because of its reference to the current state of computer science. And it fails to include some areas of potentially very large impact, namely problems that cannot now be solved well by either computer or people. To build a system to solve a particular problem, we need to do four things: 1. Define the problem precisely. This definition must include precise specifications of what the initial situation (s) will be as well as what final situations constitute acceptable solution to the problem. 2. Analyze the problem. A few very important features can have an immense impact on the appropriateness of various possible techniques for solving the problem. 3. Isolate and represent the task knowledge that is necessary to solve the problem. 4. Choose the best problem-solving techniques(s) and apply it (them) to the particular problem. Solution: Define AI Technique 1 Marks List and explain 4 thing to build system 4 Marks SCHEME [1 + 4 = 5]	5 1 4
		Explain Hill climbing Heuristic search. Answer: Is a variant of generate-and test in which feedback from the test	10

procedure is used to help the generator decide which direction to move in search space.

The test function is augmented with a heuristic function that provides an estimate of how close a given state is to the goal state.

Computation of heuristic function can be done with negligible amount of computation.

Hill climbing is often used when a good heuristic function is available for evaluating states but when no other useful knowledge is available.

Simple Hill Climbing

• Algorithm: Simple Hill Climbing

1. **Evaluate the initial state.** If it is also goal state, then return it and quit. Otherwise continue with the initial state as the current state.
 2. Loop until a solution is found or until there are no new operators left to be applied in the current state:
 - a. Select an operator that has not yet been applied to the current state and apply it to produce a new state
 - b. Evaluate the new state
 - i. If it is the goal state, then return it and quit.
 - ii. If it is not a goal state but it is better than the current state, then make it the current state.
 - iii. If it is not better than the current state, then continue in the loop.
- The key difference between Simple Hill climbing and Generate-and-test is the use of evaluation function as a way to inject task specific knowledge into the control process.
 - Is one state better than another? For this algorithm to work, precise definition of better must be provided.

3

Steepest-Ascent Hill Climbing

- This is a variation of simple hill climbing which considers all the moves from the current state and selects the best one as the next state.
- Also known as Gradient search

Algorithm: Steepest-Ascent Hill Climbing

1. **Evaluate the initial state.** If it is also a goal state, then return it and quit. Otherwise, continue with the initial state as the current state.
2. Loop until a solution is found or until a complete iteration produces no change to current state:
 - a. Let SUCC be a state such that any possible successor of the current state will be better than SUCC
 - b. For each operator that applies to the current state do:
 - i. Apply the operator and generate a new state
 - ii. Evaluate the new state. If it is a goal state, then return it and quit. If not, compare it to SUCC. If it is better, then set SUCC to this state. If it is not better, leave SUCC alone.
 - c. If the SUCC is better than the current state, then set current state to SUCC,

3

This simple policy has three well-known drawbacks:

1. **Local Maxima:** a local maximum as opposed to global maximum.
2. **Plateaus:** An area of the search space where evaluation function is flat, thus requiring random walk.

3. **Ridge:** Where there are steep slopes and the search direction is not towards the top but towards the side.

There are some ways of dealing with these problems, although these methods are by no means guaranteed:

- Backtrack to some earlier node and try going in a different direction. This is particularly reasonable if at that node there was another direction that looked as promising or almost as promising as the one that was chosen earlier. This is a fairly good way to deal with local maxima.
- Make a big jump in some direction to try to get to a new section of the search space. This is a good way of dealing with plateaus.
- Apply two or more rules before doing the test. This corresponds to moving in several directions at once. This is a good strategy for dealing with ridges.

Simulated Annealing:

Simulated annealing is a variation of hill climbing in which, at the beginning of the process, some downhill moves may be made.

The idea is to do enough exploration of the whole space early on. So that the final solution is relatively insensitive to the starting state. By doing so we can lower the chances of getting caught at local maximum, plateau or a ridge.

In this we attempt to *minimize* rather than maximize the value of the objective function. Thus this process is one of valley descending in which the object function is the energy level.

Simulated annealing as a computational process is patterned after the physical process of annealing, in which physical substances such as metals are melted and then gradually cooled until some solid state is reached.

Physical substances usually move from higher energy configurations to lower ones, so the valley descending occurs naturally.

Probability of transition to higher energy state is given by function:

$$P = e^{-\Delta E/kt}$$

Where ΔE is the positive change in the energy level T is the temperature K is Boltzmann constant.

The algorithm for simulated annealing is slightly different from the simple-hill climbing procedure. The three differences are:

- The annealing schedule must be maintained
- Moves to worse states may be accepted
- It is good idea to maintain, in addition to the current state, the best state found so far.

Algorithm: Simulate Annealing

1. Evaluate the initial state. If it is also a goal state, then return it and quit. Otherwise, continue with the initial state as the current state.

2. Initialize BEST-SO-FAR to the current state.

3. Initialize T according to the annealing schedule

4. Loop until a solution is found or until there are no new operators left to be applied in the current state.

a. Select an operator that has not yet been applied to the current state and apply it to produce a new state.

b. Evaluate the new state. Compute:

- $\Delta E = (\text{value of current}) - (\text{value of new state})$
- If the new state is a goal state, then return it and quit.
- If it is a goal state but is better than the current state, then make it the current state. Also set BEST-SO-FAR to this new state.
- If it is not better than the current state, then make it the current

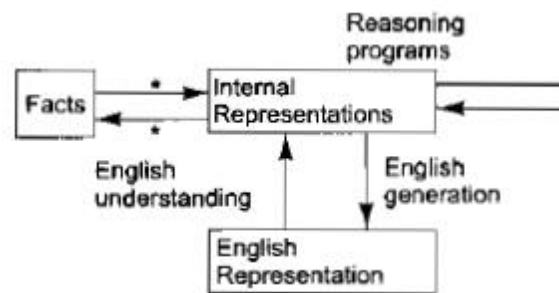


Figure 1: Mapping between Facts and Representation.

One common representation is natural language (particularly English) sentences. Regardless of the representation for facts we use in a program, we may also need to be concerned with an English representation of those facts in order to facilitate getting information into and out of the system. We need mapping functions from English sentences to the representation we actually use and from it back to sentences.

Facts

- Spot is a dog
- Dog(Spot)
the above fact
- $\forall x: dog(x) \rightarrow hastail(x)$
that

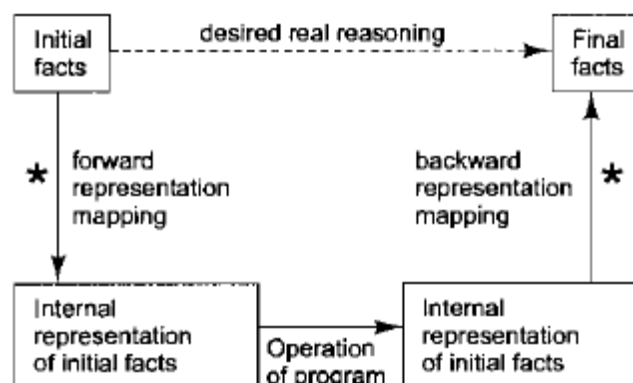
Representation

A fact represented in English
Using forward mapping function
is represented in logic.
A logical representation of the fact
“all dogs have tails”

Now using deductive mechanism we can generate a new representation of object:

- hastail(Spot)
 - Spot has a tail
to generate English
[it is new knowledge]
- A new object representation
Using backward mapping function
sentence

Below figure 2 shows an extended view of the starred part of figure 1. The dotted line across the top represents the abstract reasoning process that a program is intended to model. The solid line across the bottom represents the concrete reasoning process that a particular program performs. This program successfully models the abstract process to the extent that, when the backward representation mapping is applied to the program's output, the appropriate final facts are actually generated.



as that shown in our example. In particular, it is possible to augment the basic representation with inference mechanisms that operate on the structure of the representation. One of the most useful forms of inference is property inheritance, in which elements of specific classes inherit attributes and values from more general classes in which they are included.

In order to support property inheritance, objects must be organized into classes and classes must be arranged in a generalization hierarchy. Figure below shows some additional baseball knowledge inserted into a structure that is so arranged. Lines represent attributes. Boxed nodes represent objects and values of attributes of objects.

Baseball knowledge

- *isa* : show class inclusion

- *instance* : show class membership

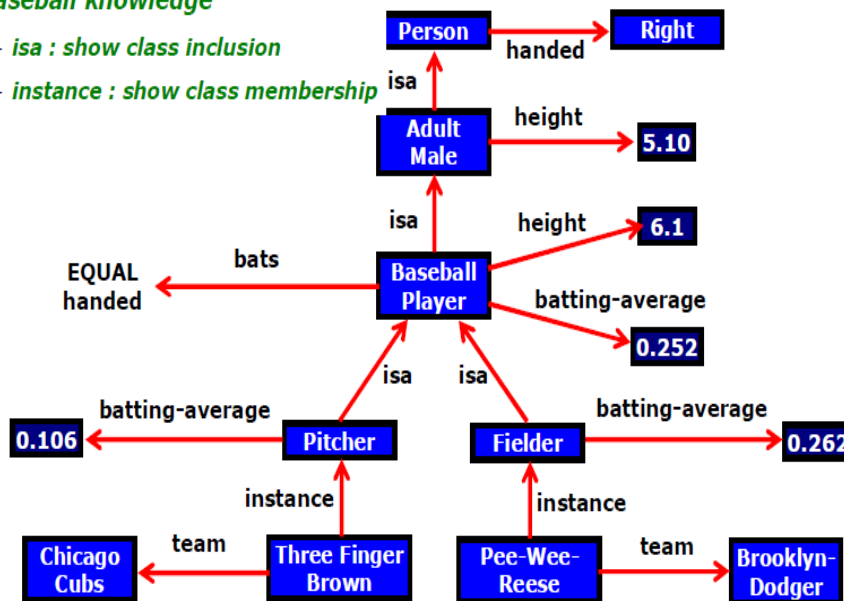


Figure 1: Inheritable Knowledge

Figure below shows the node for baseball player displayed as a frame.

Baseball-Player

isa:	Adult-Male
bats:	(EQUAL handed)
height:	6-1
batting-average:	.252

Figure 2: Viewing a Node as a Frame

An idealized form of the property inheritance algorithm can be used as follows:

Algorithm: Property Inheritance

To retrieve a value V for an attribute of an instance object O:

1. Find the O object in the knowledge base
2. If there is a value for the attribute A, report that value.
3. Otherwise, see if there is a value for the attribute instance. If not, then fail
4. Otherwise, move to that node corresponding to that value and look for a value for the attribute A. If one is found, report it
5. Otherwise, do until there is no value for the isa attribute or until an answer is found.
 - (a) Get the value of the isa attribute and move to that node.
 - (b) See if there is a value for the attribute A. If there is, report it.

This algorithm is simple,

‡ It does describe the basic mechanism of inheritance.

	‡ It does not say what to do if there is more than one value of the instance or “isa” attribute. Solution: List different types of Knowledge 2 Marks Explain Inheritable Knowledge 2 Marks Example diagram 2 Marks Property Inheritance Algorithm 2 Marks SCHEME [2 + 2 + 2 + 2 = 8]	
B	What are the issues in Knowledge Representation? Answer: The fundamental goal of Knowledge Representation is to facilitate inferencing (conclusions) from knowledge. The issues that arise while using KR techniques are many. Some of these are explained below. ◇ Important Attributes: Any attribute of objects so basic that they occur in almost every problem domain?. If there are, we need to make sure that they are handled appropriately in each of the mechanisms we propose. If such attribute exist, what are they? ◇ Relationship among attributes: Any important relationship that exists among object attributes? ◇ Choosing Granularity: At what level of detail should the knowledge be represented? Is there a good set of primitives into which all knowledge can be broken down? Is it helpful to use such primitives? ◇ Set of objects: How sets of objects be represented? ◇ Finding Right structure: Given a large amount of knowledge stored, how can relevant parts be accessed when they are needed? Solution: List five different issues 7 Marks SCHEME [7 + 7 = 7]	7